

Software Factory and CI/CD

Version 1, 2026-08-05

Software Factory and DevSpaces

Introduction

Welcome to this quick course on the **Software Factory and CI/CD**. This course is the **third** in a series of **six** courses about **Application Platform Delivery**:

- [Application Platform Delivery Foundations](#)
- [Software Factory & Developer Experience](#)
- [Software Factory and DevSpaces\(This course\)](#)
- [Software Factory and CICD](#)
- [Application Modernization](#)
- [Trusted Software Supply Chain \(TSSC\)](#)

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 2.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

CI with OpenShift Pipelines

```
<div class="module-card">  
<div class="module-header">
```



What's in this section?

```
</div>
```

In this section, you'll build a complete cloud-native Continuous Integration (CI) workflow using OpenShift Pipelines (Tekton).

You'll create a reusable CI pipeline, securely retrieve registry credentials from HashiCorp Vault, automate pipeline execution with GitLab webhooks and Tekton Triggers, and validate the end-to-end build process that produces container images for deployment on OpenShift.

```
</div>
```

Lab 1: CI Pipeline

In this lab, you'll prepare the foundational resources required for Tekton pipelines.

You'll create a shared workspace for pipeline tasks and configure a ServiceAccount with the permissions required to build container images and deploy application resources.

By the end of this lab, you will have:

- Created a PersistentVolumeClaim for the pipeline workspace
- Created a ServiceAccount for running Tekton pipelines
- Granted the ServiceAccount permissions to manage resources in the development project

Create the Pipeline Workspace and Service Account

Tekton pipelines use a shared workspace to exchange files between tasks. In this exercise, you'll create a PersistentVolumeClaim (PVC) that provides persistent storage for the pipeline, along with a ServiceAccount that Tekton uses to execute pipeline tasks.

1. Open the **OpenShift Console URL** listed on the **RHDP Lab Environment** page.
2. Sign in using any one of the credentials listed in the **Lab Credentials** section.
3. After signing in, open the **Developer** perspective.
4. Click the **>_ (Web Terminal)** icon in the upper-right corner to launch a terminal session.
5. Create the pipeline workspace and ServiceAccount by running the following command:

```
oc apply -f - <<'EOF'
```

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipeline-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline
  namespace: etx-app-dev
EOF

```

The command creates:

- A **pipeline-workspace-pvc** PersistentVolumeClaim for sharing files between pipeline tasks.
- A **pipeline** ServiceAccount that Tekton uses to execute the pipeline.



If the **pipeline** ServiceAccount already exists (for example, it was created during namespace provisioning), you may see **serviceaccount/pipeline configured** instead of **created**. This is expected and can be safely ignored.

Grant Pipeline Permissions

The pipeline ServiceAccount requires permission to create and manage application resources within the development project.

1. Run the following command:

```

oc adm policy add-role-to-user edit \
  system:serviceaccount:etx-app-dev:pipeline \
  -n etx-app-dev

```

The **edit** role allows the pipeline to create and manage resources such as ImageStreams, Deployments, Services, Routes, and other application resources in the **etx-app-dev** project.

Verify the Pipeline Resources

Confirm that the workspace and ServiceAccount were created successfully.

```

oc get pvc pipeline-workspace-pvc

```

```
oc get sa pipeline
```

Lab 2: Create the CI Pipeline

In this lab, you'll create a Tekton CI pipeline that builds the application, retrieves container registry credentials from HashiCorp Vault, and produces a container image ready for deployment.

By the end of this lab, you will have:

- Created a custom Tekton Task to retrieve registry credentials from Vault
- Built a reusable Tekton Pipeline for application builds
- Configured the pipeline to clone source code, build the application, and build a container image

Create a Task to Retrieve Registry Credentials

The CI pipeline requires credentials to authenticate with the container registry before it can push application images.

In this section, you'll create a custom Tekton Task that authenticates to HashiCorp Vault using the Kubernetes authentication method configured in the previous supporting services lab, retrieves the registry credentials, and makes them available to subsequent pipeline tasks.

First, retrieve the Vault route from the OpenShift cluster.

```
VAULT_URL="https://$(oc get route \
  -n vault \
  -o jsonpath='{.items[0].spec.host}')"

echo "${VAULT_URL}"
```

Next, create the Tekton Task.

```
oc apply -f - <<EOF
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-registry-creds
  namespace: etx-app-dev
spec:
  workspaces:
    - name: source
  results:
    - name: registry-server
    - name: registry-username
    - name: registry-password
  steps:
```

```

- name: fetch
  image: registry.access.redhat.com/ubi9/ubi-minimal:latest
  script: |
    #!/bin/sh
    set -e

    microdnf install -y jq && microdnf clean all

    VAULT_ADDR="${VAULT_URL}"
    SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

    VAULT_TOKEN=$(curl -s --request POST \
      "${VAULT_ADDR}/v1/auth/kubernetes/login" \
      --data "{\"role\":\"pipeline\",\"jwt\":\"${SA_TOKEN}\"}" \
      | jq -r '.auth.client_token')

    CREDS=$(curl -s \
      --header "X-Vault-Token: ${VAULT_TOKEN}" \
      "${VAULT_ADDR}/v1/secret/data/registry")

    echo "${CREDS}" | jq -r '.data.data.server' \
      | tr -d '\n' > $(results.registry-server.path)

    echo "${CREDS}" | jq -r '.data.data.username' \
      | tr -d '\n' > $(results.registry-username.path)

    echo "${CREDS}" | jq -r '.data.data.password' \
      | tr -d '\n' > $(results.registry-password.path)
EOF

```



This Task authenticates to Vault using the ServiceAccount token mounted into the pipeline pod. It retrieves the registry server, username, and password from Vault and exposes them as Tekton Task results for use by downstream pipeline tasks.

Create the CI Pipeline

Next, create the Tekton Pipeline that orchestrates the complete CI workflow.

The pipeline performs the following activities:

- Clones the application source code from Git
- Builds and tests the application using Maven
- Retrieves registry credentials from Vault
- Builds the application container image using Buildah

Create the pipeline.

```
oc apply -f - <<'EOF'
```

```

apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-base-app-ci
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      type: string
    - name: git-revision
      type: string
      default: main
    - name: image-name
      type: string
  workspaces:
    - name: shared-workspace
  tasks:
    - name: clone
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: git-clone
          - name: namespace
            value: openshift-pipelines
        params:
          - name: URL
            value: $(params.git-url)
          - name: REVISION
            value: $(params.git-revision)
        workspaces:
          - name: output
            workspace: shared-workspace

    - name: build-and-test
      runAfter: [clone]
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: maven
          - name: namespace
            value: openshift-pipelines
        params:
          - name: GOALS
            value: ["package", "-DskipTests=false"]
        workspaces:

```

```

- name: source
  workspace: shared-workspace

- name: fetch-creds
  runAfter: [clone]
  taskRef:
    name: vault-fetch-registry-creds
  workspaces:
    - name: source
      workspace: shared-workspace

- name: build-image
  runAfter: [build-and-test, fetch-creds]
  taskRef:
    resolver: cluster
    params:
      - name: kind
        value: task
      - name: name
        value: buildah
      - name: namespace
        value: openshift-pipelines
  params:
    - name: IMAGE
      value: $(params.image-name):$(params.git-revision)
    - name: DOCKERFILE
      value: Containerfile
  workspaces:
    - name: source
      workspace: shared-workspace

```

EOF



The `git-clone`, `maven`, and `buildah` Tasks are installed by the OpenShift Pipelines Operator in the `openshift-pipelines` namespace. This pipeline references them using the cluster resolver, which replaces the deprecated `ClusterTask` resource. You can verify that these Tasks are available by running:

```
oc get task -n openshift-pipelines
```

Lab 3: Test CI Pipeline

In this lab, you'll manually execute the CI pipeline to verify that it successfully clones the application source code, builds the application, retrieves registry credentials from Vault, and pushes the container image to the internal OpenShift registry.

You'll also learn how to monitor pipeline execution and troubleshoot common issues before configuring automated pipeline triggers.

By the end of this lab, you will have:

- Created a PipelineRun to execute the CI pipeline manually
- Monitored the execution of PipelineRuns and TaskRuns
- Verified that the application image was successfully built and pushed
- Learned how to troubleshoot common pipeline failures

Run the CI Pipeline

Before configuring Git webhooks, verify that the pipeline executes successfully by creating a manual PipelineRun.

1. Open the **OpenShift Console URL** listed on the **RHDP Lab Environment** page.
2. Sign in using any one of the credentials listed in the **Lab Credentials** section.
3. After signing in, open the **Developer** perspective.
4. Click the >_ (**Web Terminal**) icon in the upper-right corner to launch a terminal session.
5. Retrieve the GitLab route by running:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route \
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: ${GITLAB_URL}"
```

6. Create a PipelineRun:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-base-app-ci-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-base-app-ci
  params:
    - name: git-url
      value: ${GITLAB_URL}/software-factory/etx_app_base_app
    - name: git-revision
      value: main
    - name: image-name
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app
  workspaces:
```

```
- name: shared-workspace
  persistentVolumeClaim:
    claimName: pipeline-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

The PipelineRun starts the CI pipeline using the source code in GitLab and executes each configured task.

Monitor Pipeline Execution

Once the PipelineRun starts, monitor its progress to ensure each task completes successfully.

Watch the PipelineRun Status

The simplest way to monitor execution is to watch the PipelineRun status in real time.

```
oc get pipelinerun -n etx-app-dev -w
```

When the pipeline finishes successfully, its status changes to **Succeeded**.

View TaskRun Details

To inspect the individual tasks executed by the pipeline:

```
LATEST_PR=$(oc get pipelinerun \
  -n etx-app-dev \
  --sort-by=.metadata.creationTimestamp \
  -o jsonpath='{.items[-1].metadata.name}')

echo "PipelineRun: ${LATEST_PR}"

oc get taskrun \
  -n etx-app-dev \
  -l tekton.dev/pipelineRun=${LATEST_PR}
```

To view the logs for a specific TaskRun:

```
oc logs \
  -n etx-app-dev \
  -l tekton.dev/taskRun=<taskrun-name> \
  -f
```



Replace **<taskrun-name>** with one of the TaskRun names returned by the previous command.

Verify the Container Image

After the pipeline completes, verify that the container image was pushed successfully.

```
oc get imagestream etx-app-base-app -n etx-app-dev  
  
oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

If both resources exist, the image was successfully built and pushed to the internal OpenShift image registry.

Troubleshoot Common Issues

If the pipeline does not complete successfully, the following checks can help identify common problems.

Vault Authentication Errors

If the `fetch-creds` task fails with an error similar to:

```
parse error: Invalid numeric literal
```

Verify that Vault is running:

```
oc get pods -n vault
```

Also confirm that:

- The Kubernetes authentication method is configured in Vault.
- The `pipeline` ServiceAccount exists in the `etx-app-dev` project.

```
oc get serviceaccount pipeline -n etx-app-dev
```

Git Clone Errors

If the `git-clone` task cannot resolve the GitLab host, verify the GitLab route:

```
oc get route \  
  -n gitlab \  
  -l app=webservice \  
  -o jsonpath='{.items[0].spec.host}{"\n"}'
```

If necessary, recreate the PipelineRun using the correct GitLab URL.

Image Push Verification

If the pipeline reports success but the image cannot be found, verify the ImageStream:

```
oc get imagestream etx-app-base-app -n etx-app-dev  
  
oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

A successful pipeline should create both resources in the **etx-app-dev** project.

Lab 4: Automate CI Pipeline

In this lab, you'll configure Tekton Triggers and GitLab webhooks to automatically start the CI pipeline whenever changes are pushed to the application repository.

You'll create the Tekton Trigger resources, expose an EventListener, configure GitLab to send push events, and verify that commits automatically trigger new PipelineRuns.

By the end of this lab, you will have:

- Created a TriggerBinding to map GitLab webhook payloads to pipeline parameters
- Created a TriggerTemplate that generates PipelineRuns
- Deployed an EventListener to receive GitLab webhook events
- Exposed the EventListener using an OpenShift Route
- Configured a GitLab webhook to trigger the pipeline automatically
- Verified end-to-end CI pipeline automation

Create the Tekton Trigger Resources

Tekton Triggers connect external events, such as GitLab push notifications, to Tekton Pipelines.

In this exercise, you'll create the TriggerBinding, TriggerTemplate, and EventListener resources required to automatically start a new PipelineRun whenever code is pushed to GitLab.

Create the TriggerBinding

The TriggerBinding extracts values from the GitLab webhook payload and maps them to parameters used by the pipeline.

1. In the Web Terminal, run:

```
<existing TriggerBinding YAML>
```

The binding maps:

- The Git repository URL

- The commit SHA used as the pipeline revision

Create the TriggerTemplate

The TriggerTemplate defines the PipelineRun that Tekton creates when a webhook event is received.

1. Run the following command:

```
<existing TriggerTemplate YAML>
```

The template creates a new PipelineRun using the shared workspace and pipeline ServiceAccount created in the previous labs.

Create the EventListener

The EventListener exposes an HTTP endpoint that receives webhook requests from GitLab.

1. Run:

```
<existing EventListener YAML>
```

The EventListener connects the GitLab webhook to the TriggerBinding and TriggerTemplate.

Expose the EventListener

GitLab must be able to securely reach the EventListener.

1. Create an HTTPS route:

```
<existing route command>
```

2. Retrieve the EventListener URL:

```
<existing echo command>
```

Copy this URL. You'll use it when configuring the GitLab webhook.



GitLab requires HTTPS endpoints for webhooks. The OpenShift Route provides TLS termination for the EventListener.

Configure the GitLab Webhook

To allow GitLab to automatically trigger the pipeline, configure a webhook for the application repository.



Creating GitLab webhooks requires **Maintainer** or **Owner** access to the project. Administrative configuration also requires the GitLab **root** account.

Enable Local Network Webhooks

By default, GitLab blocks webhook requests to internal cluster addresses.

1. Retrieve the GitLab administrator information:

```
<existing command>
```

2. Sign in to GitLab using the built-in **root** administrator account.
3. Navigate to:
 - **Admin Area** › **Settings** › **Network**
4. Expand **Outbound requests**.
5. Enable:
 - ☒ Allow requests to the local network from webhooks and integrations
6. Save the changes.



This setting allows GitLab to communicate with services running inside the OpenShift cluster, including the Tekton EventListener.

Create the Repository Webhook

After enabling local network requests, configure the repository webhook.

1. Open the **etx_app_base_app** repository.
2. Navigate to:
 - **Settings** › **Webhooks**
3. Click **[Add new webhook]**.
4. Configure the webhook:
 - URL: Paste the EventListener Route URL
 - Trigger: Push events
 - SSL Verification: Disable for this lab environment
5. Click **[Add webhook]**.

Test the Webhook

Verify that GitLab can communicate with the EventListener.

1. Select the webhook.
2. Click **[Test]** → **Push events**.

A successful test returns an HTTP **202 Accepted** response.

Verify Automatic Pipeline Execution

After the webhook is configured successfully, verify that GitLab automatically starts the CI pipeline.

1. Commit and push a small change to the **main** branch.
2. Monitor PipelineRuns:

```
oc get pipelinerun -n etx-app-dev -w
```

A new PipelineRun should be created automatically without manual intervention.

Troubleshoot Common Issues

If the webhook does not trigger the pipeline, verify the following.

Invalid URL

If GitLab reports **Invalid url given**:

- Ensure local network webhooks are enabled.
- Verify the EventListener Route uses HTTPS.
- Confirm the Route is reachable.

SSL Certificate Errors

If webhook delivery fails due to certificate validation:

- Disable SSL verification for the webhook.

This is acceptable for the lab environment because the cluster uses self-signed certificates.

EventListener Not Receiving Requests

Verify that the EventListener pod is running:

```
oc get pods -n etx-app-dev | grep el-gitlab-listener  
  
oc logs -n etx-app-dev -l eventlistener=gitlab-listener
```

The logs should show incoming webhook requests from GitLab.

Lab 5: Validate CI Pipeline

In this lab, you'll verify that your GitLab webhook automatically triggers the Tekton pipeline whenever code is pushed to the application repository.

You'll make a small change to the application, observe the pipeline execution, and confirm that a new container image is successfully built and stored in the OpenShift internal registry.

By the end of this lab, you will have:

- Triggered the CI pipeline automatically using a Git push
- Monitored the PipelineRun execution
- Verified that the application image was successfully built and pushed
- Confirmed that the CI pipeline is fully operational

Trigger the CI Pipeline

Now that the webhook integration is configured, any push to the GitLab repository automatically starts the Tekton pipeline.

1. Open your **etx-app-base-app** Dev Spaces workspace.
2. Open a terminal in the workspace.
3. Make a small change and push it to the Git repository.

```
# Make a trivial change
echo "# CI/CD test" >> README.md

git add README.md
git commit -m "Test webhook trigger"
git push origin feature/add-devfile
```

The Git push sends a webhook event to the Tekton EventListener, which automatically creates a new PipelineRun.

Monitor the Pipeline Execution

Verify that a new pipeline has started.

1. Run the following command:

```
oc get pipelinerun -n etx-app-dev
```

You should see a new **etx-app-base-app-ci-*** PipelineRun with a status of **Running**. Once complete, the status changes to **Succeeded**.

View Pipeline Logs

Monitor the execution of the latest PipelineRun.

1. Retrieve the most recent PipelineRun and stream its logs.

```
# Get the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev \
  --sort-by=.metadata.creationTimestamp \
  -o jsonpath='{.items[-1].metadata.name}')

echo "Following logs for: $LATEST_PR"

# Stream logs from the currently running task
oc logs -f $(oc get pods \
  -n etx-app-dev \
  -l tekton.dev/pipelineRun=$LATEST_PR \
  --sort-by=.metadata.creationTimestamp \
  -o jsonpath='{.items[-1].metadata.name}') \
  -n etx-app-dev
```

Review the output to confirm that all pipeline tasks complete successfully.



Since Tekton executes each task in a separate pod, the log stream ends when the current task finishes. If needed, rerun the command to follow the next task.

Verify the Generated Image

Confirm that the pipeline pushed the application image to the OpenShift internal registry.

1. Run the following command:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

Verify that the ImageStream contains a newly generated image tagged with the latest commit SHA.

Verify the CI Environment

Perform a final validation of the pipeline components.

1. Run the following commands:

```
# Verify the pipeline
oc get pipeline etx-app-base-app-ci -n etx-app-dev

# Verify the EventListener
oc get pods -n etx-app-dev | grep el-gitlab-listener

# Verify the EventListener route
oc get route el-gitlab-listener -n etx-app-dev

# Display recent PipelineRuns
oc get pipelinerun -n etx-app-dev \
```

```
--sort-by=.metadata.creationTimestamp | tail -5
```

Confirm that:

- The `etx-app-base-app-ci` pipeline exists.
- The `el-gitlab-listener` pod is running.
- The EventListener route is available.
- The latest PipelineRun completed successfully.



If the pipeline does not execute as expected, check the following:

- **Pipeline fails during clone** — Verify the Git repository URL and credentials.
- **Build task fails** — Ensure the application builds successfully outside the pipeline.
- **Image push fails** — Confirm the `pipeline` ServiceAccount has the required permissions.
- **Pipeline is not triggered** — Verify the GitLab webhook configuration and ensure the EventListener pod is running.

Lab Summary

Throughout these labs, you built a complete continuous integration (CI) workflow using OpenShift Pipelines (Tekton).

Starting with the foundational pipeline resources, you created a reusable CI pipeline that clones application source code, builds and tests the application, securely retrieves registry credentials from HashiCorp Vault, and publishes container images to the OpenShift internal registry. You then automated pipeline execution by integrating GitLab webhooks with Tekton Triggers and verified the complete end-to-end CI workflow.

You now have:

- A Tekton pipeline for building, testing, and containerizing applications
- A shared pipeline workspace and ServiceAccount with the required permissions
- Secure registry credential retrieval using HashiCorp Vault
- Automated pipeline execution through GitLab webhooks and Tekton Triggers
- Container images automatically published to the OpenShift internal registry
- A fully validated, event-driven CI pipeline running on OpenShift

GitOps Continuous Deployment

```
<div class="module-card">
<div class="module-header">
```



What's in this section?

```
</div>
```

In this section, you'll configure Argo CD to authenticate using the shared Keycloak OpenID Connect (OIDC) integration, create a GitOps repository to store Kubernetes manifests, and deploy the application using automated synchronization. Finally, you'll connect the Tekton CI pipeline to the GitOps repository so that every successful build automatically updates the application deployment.



You will deploy your **own** Argo CD instance in the **etx-app-dev** namespace. The pre-installed OpenShift GitOps instance in the **openshift-gitops** namespace is used for platform infrastructure and is independent of the application GitOps workflow you'll build in this section.

```
</div>
```

Lab 6: GitOps Authentication

In this lab, you'll verify that the OpenShift GitOps Operator is installed and configure OpenID Connect (OIDC) authentication for Argo CD using the shared Keycloak client.

By the end of this lab, you will have:

- Verified that the OpenShift GitOps Operator is installed
- Configured Argo CD to use the shared Keycloak OIDC client
- Created the OIDC secret required for Argo CD authentication

Verify the OpenShift GitOps Operator

The OpenShift GitOps Operator is pre-installed as part of the lab environment. Before creating GitOps resources, verify that the operator is available and running successfully.

1. Open the **OpenShift Console URL** listed on the **RHDP Lab Environment** page.
2. Sign in using any one of the credentials listed in the **Lab Credentials** section.
3. After signing in, open the **Developer** perspective.
4. Click the **>_ (Web Terminal)** icon in the upper-right corner to launch a terminal session.
5. Verify that the OpenShift GitOps Operator is installed:

```
oc get csv -n openshift-operators | grep gitops
```

Confirm that the **openshift-gitops-operator** ClusterServiceVersion (CSV) is listed with the **Succeeded** phase.



If the operator is not yet in the **Succeeded** phase, wait a few moments and rerun the command before proceeding.

Configure Keycloak Authentication for Argo CD

The ETX platform uses a shared Keycloak OpenID Connect (OIDC) client named **argocd** for all Argo CD instances. This enables users to authenticate to Argo CD using the same enterprise credentials they use for GitLab, Vault, and other platform services.

In this exercise, you'll create the Kubernetes Secret that stores the shared OIDC client credentials.

1. Run the following command to create the Argo CD OIDC secret:

```
# Get Keycloak URL (etx-sso route)
KEYCLOAK_URL="https://etx-sso.apps. $(oc get route -n keycloak -o
jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//')"
```

```
# Shared Argo CD client secret
CLIENT_SECRET="changeme"
```

```
oc create secret generic argocd-oidc-secret \
  -n etx-app-dev \
  --from-literal=clientSecret=${CLIENT_SECRET}
```

```
# Label the secret for the Argo CD Operator
oc label secret argocd-oidc-secret \
  -n etx-app-dev \
  app.kubernetes.io/name=argocd-oidc-secret \
  app.kubernetes.io/part-of=argocd
```

```
echo "Keycloak issuer: ${KEYCLOAK_URL}/realms/etx-lab"
```

The command creates the **argocd-oidc-secret** Secret in the **etx-app-dev** namespace and labels it so it can be used by the OpenShift GitOps Operator.



The **argocd** OIDC client is shared across the ETX platform. Do **not** create a new client in Keycloak. Use the existing client and the shared client secret provided for this lab.

Lab 7: Deploy Argo CD

In this lab, you'll deploy an Argo CD instance in your development project and configure it to use the shared Keycloak OpenID Connect (OIDC) integration created in the previous lab.

By the end of this lab, you will have:

- Deployed an Argo CD instance in the `etx-app-dev` project
- Configured Argo CD to authenticate using the shared Keycloak OIDC integration
- Verified that the Argo CD components and Route are available
- Accessed the Argo CD dashboard using Keycloak Single Sign-On

Deploy the Argo CD Instance

In this exercise, you'll deploy an Argo CD instance that will be used to manage application deployments throughout the remaining labs.

1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Retrieve the cluster domain:

```
# Get cluster domain from the existing Keycloak route
CLUSTER_DOMAIN="$(oc get route -n keycloak \
  -o jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//')"
```

```
echo "Cluster domain: ${CLUSTER_DOMAIN}"
```

3. Deploy the Argo CD instance:

```
CLUSTER_DOMAIN="$(oc get route -n keycloak \
  -o jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//')"
```

```
cat <<EOF | oc apply -f -
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: argocd
  namespace: etx-app-dev
spec:
  server:
    autoscale:
      enabled: false
    grpc:
      ingress:
        enabled: false
    ingress:
      enabled: false
```

```

insecure: true
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 125m
    memory: 128Mi
route:
  enabled: true
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
service:
  type: ClusterIP
grafana:
  enabled: false
  ingress:
    enabled: false
  route:
    enabled: false
prometheus:
  enabled: false
  ingress:
    enabled: false
  route:
    enabled: false
initialSSHKnownHosts: {}
rbac:
  defaultPolicy: 'role:readonly'
  policy: |
    g, platform-admin, role:admin
    g, platform-developer, role:admin
  scopes: '[groups, preferred_username]'
extraConfig:
  oidc.config: |
    name: ETX Keycloak
    issuer: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab
    clientID: argocd
    clientSecret: $argocd-oidc-secret:clientSecret
    requestedScopes:
      - openid
    requestedIDTokenClaims:
      groups:
        essential: true
    logoutURL: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab/protocol/openid-connect/logout
repo:
  resources:
    limits:
      cpu: 1000m

```

```

        memory: 1Gi
      requests:
        cpu: 250m
        memory: 256Mi
    resourceExclusions: |
      - apiGroups:
        - tekton.dev
        clusters:
        - '*'
        kinds:
        - TaskRun
        - PipelineRun
    controller:
      processors: {}
      resources:
        limits:
          cpu: 2000m
          memory: 2Gi
        requests:
          cpu: 250m
          memory: 1Gi
    ha:
      enabled: false
      resources:
        limits:
          cpu: 500m
          memory: 256Mi
        requests:
          cpu: 250m
          memory: 128Mi
EOF

```

The command deploys an Argo CD instance named `argocd` in the `etx-app-dev` project.



The `server.insecure: true` setting configures Argo CD to serve HTTP traffic internally while the OpenShift Route performs TLS termination. This prevents redirect loops (`ERR_TOO_MANY_REDIRECTS`) when using edge TLS termination.



The Argo CD custom resource references the `argocd-oidc-secret` created in the previous lab, enabling authentication through the shared Keycloak OIDC client.

Verify the Argo CD Deployment

Wait for the Argo CD components to start.

1. Run the following command:

```
oc get pods -n etx-app-dev -w
```

Wait until all `argocd-*` pods report a status of `1/1 Ready`, then press `Ctrl + C` to stop watching.

Once all pods are running, your Argo CD instance is ready to manage GitOps applications in the following labs.

Verify Route Creation

The Argo CD server is exposed through an OpenShift Route, allowing you to access the web dashboard from your browser. In this exercise, you'll verify that the route exists and retrieve the dashboard URL.

1. Verify that the Argo CD route has been created:

```
oc get route argocd-server -n etx-app-dev
```

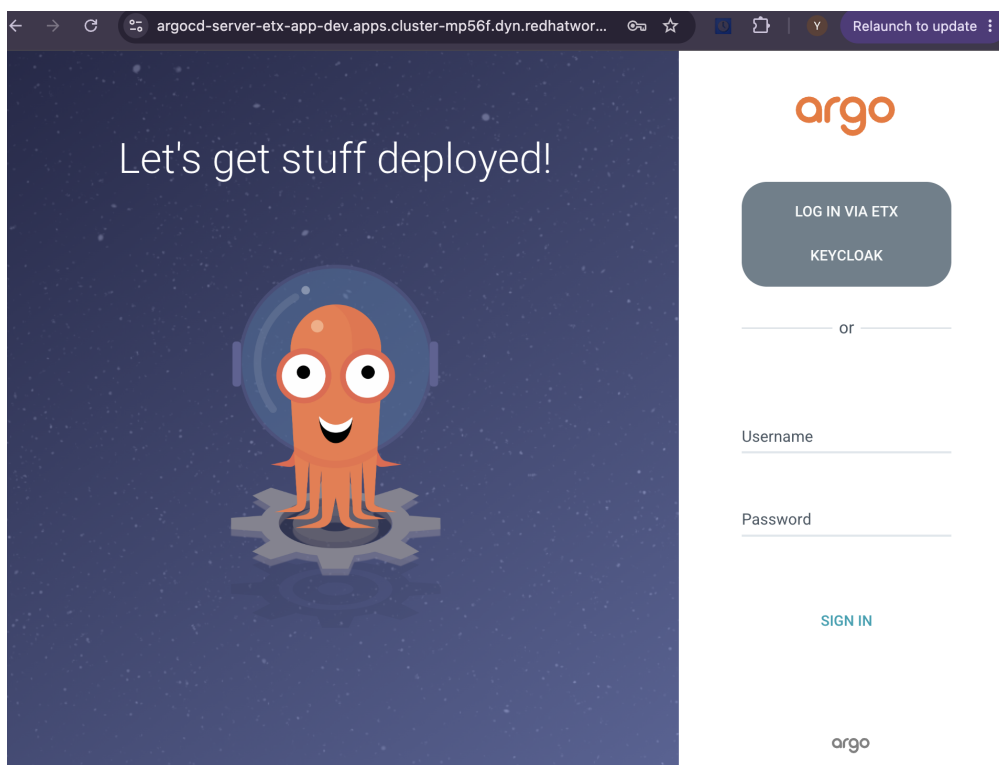


Figure 1. Sample Argo CD Route

If the route does not exist, create it manually:

```
oc create route edge argocd-server \
  --service=argocd-server \
  --port=http \
  --insecure-policy=Redirect \
  -n etx-app-dev
```

Access the Argo CD Dashboard

Retrieve the Argo CD dashboard URL and sign in to verify that your deployment is working correctly.

1. Get the Argo CD URL:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}')"

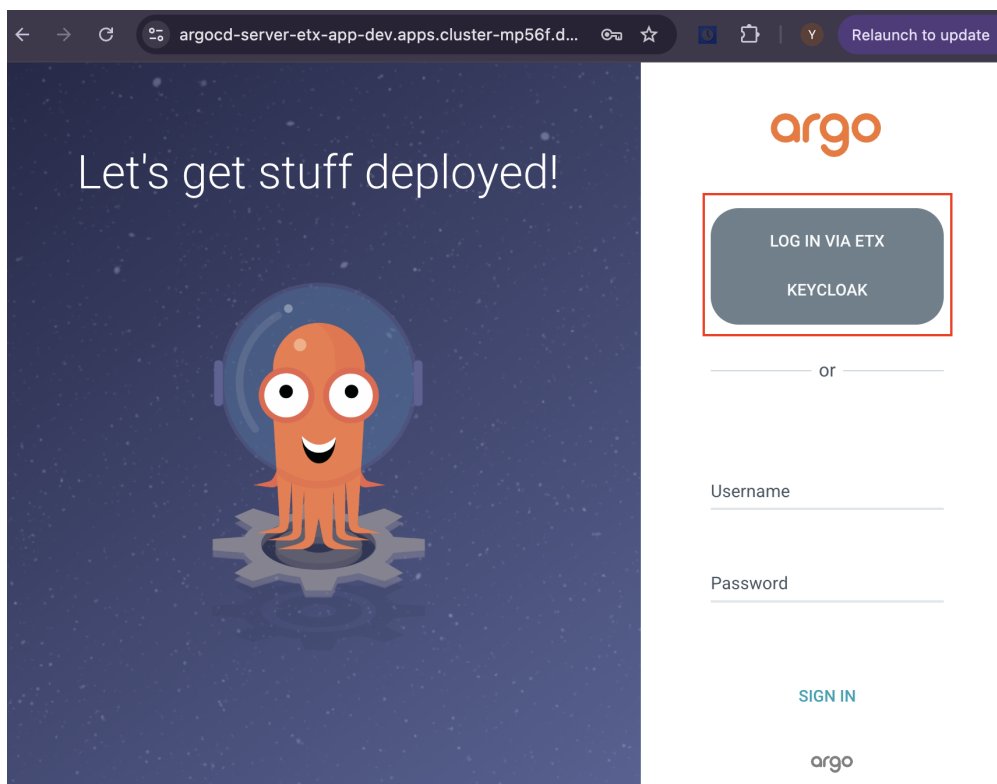
```

2. Open the displayed URL in your browser.

You can authenticate using either ETX Keycloak Single Sign-On or the built-in Argo CD administrator account.

Sign in with ETX Keycloak (Recommended)

1. On the Argo CD login page, select **ETX Keycloak**.



2. Sign in using one of the lab credentials:
 - `platform-admin / openshift`
 - `platform-developer / openshift`
3. After successful authentication, you'll be redirected to the Argo CD dashboard.



Keycloak Single Sign-On provides the same authentication experience used throughout the ETX platform, including GitLab, Quay, and Vault.

Sign in with the Admin Account (Optional)

If you prefer to use the built-in administrator account:

1. Retrieve the default admin password:

```
oc get secret argocd-cluster -n etx-app-dev \
-o jsonpath='{.data.admin\.password}' | base64 -d && echo
```

2. Sign in using:

- **Username:** `admin`
- **Password:** Output from the previous command

Troubleshooting Dashboard Access

If you receive an `ERR_TOO_MANY_REDIRECTS` error when opening the Argo CD dashboard, verify that the Argo CD server is configured to run in insecure mode behind the OpenShift TLS-terminating route.

1. Confirm the current setting:

```
oc get argocd argocd -n etx-app-dev \
-o jsonpath='{.spec.server.insecure}{"\n"}'
```

The command should return:

```
true
```

2. If the value is not `true`, update the Argo CD instance:

```
oc patch argocd argocd -n etx-app-dev --type=merge \
-p '{"spec":{"server":{"insecure":true}}}'
```

3. Wait for the Argo CD server pod to restart:

```
oc get pods -n etx-app-dev \
-l app.kubernetes.io/name=argocd-server -w
```

Press `Ctrl + C` once the new pod reports `1/1 Ready`.



If the redirect error persists after the pod restarts, clear your browser cache or open the dashboard in an incognito/private browsing window before trying again.

Lab 8: GitOps Repository

In this lab, you'll create a GitOps repository in GitLab to store the Kubernetes manifests that define your application deployment. This repository will serve as the source of truth that Argo CD continuously monitors and synchronizes with your OpenShift cluster.

By the end of this lab, you will have:

- Created a GitOps repository in GitLab
- Organized Kubernetes manifests for the development environment
- Committed and pushed the initial application manifests

Create the GitOps Repository

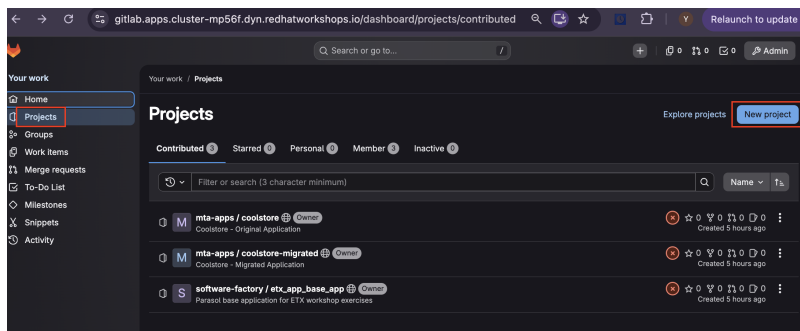
In this exercise, you'll create a new GitLab repository that will store the Kubernetes manifests used by Argo CD for GitOps-based application deployment.

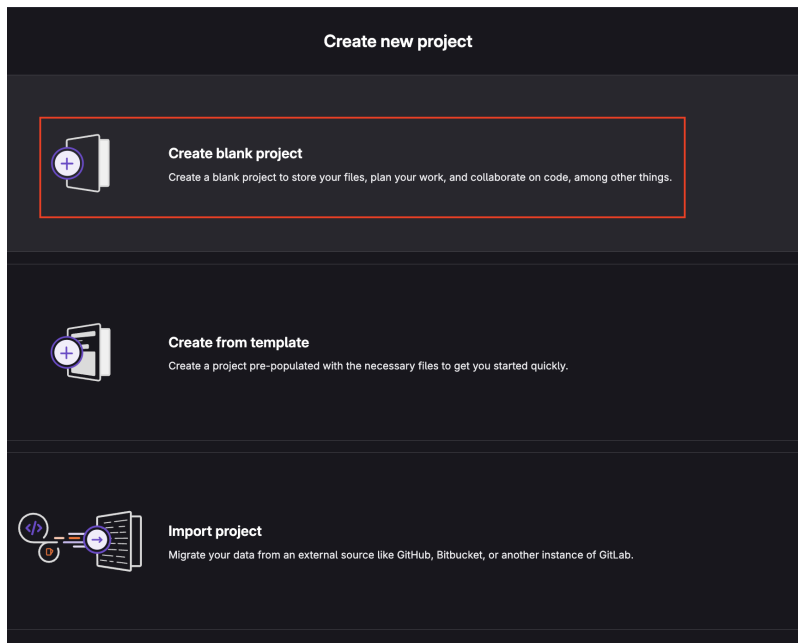
1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Retrieve the GitLab URL:

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "GitLab URL: ${GITLAB_URL}"
```

3. Open the displayed GitLab URL in your browser.
4. Create a new project named **etx-app-gitops** in the **software-factory** group.





Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name
etx-app-gitops

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL
https://gitlab.apps.cluster-mp56f.dyn.redhatworkshops.io/ software-factory

Project slug
etx-app-gitops

Visibility Level

- ☐ Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.
- ☐ Internal
The project can be accessed by any logged in user except external users.
- ☒ Public
The project can be accessed without any authentication.

Project Configuration

- ☒ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.
- ☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. [Learn more.](#)
- ☐ Enable Secret Detection
Scan your code for secrets and credentials to prevent unauthorized access. [Learn more.](#)

[Create project](#) [Cancel](#)

5. Clone the repository locally:

```
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git  
cd etx-app-gitops
```

The repository will use the following directory structure:

```

etx-app-gitops/
├── environments/
│   ├── dev/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── route.yaml
│   └── stage/
│       ├── namespace.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       └── route.yaml
└── README.md

```

Create the Development Environment Manifests

Create the Kubernetes manifests that define the application's development environment.

1. Create the directory structure and namespace manifest:

```

mkdir -p environments/dev

cat > environments/dev/namespace.yaml <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-dev
EOF

```

2. Create the Deployment manifest:

```

cat > environments/dev/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:

```

```

    labels:
      app: etx-app-base-app
    annotations:
      instrumentation.opentelemetry.io/inject-java: "true"
  spec:
    containers:
      - name: etx-app-base-app
        image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app:latest
        ports:
          - containerPort: 8080
        resources:
          requests:
            cpu: 250m
            memory: 512Mi
          limits:
            cpu: "1"
            memory: 1Gi
EOF

```



The OpenTelemetry annotation automatically injects the Java agent using the Instrumentation custom resource created during the supporting services setup.



The container image tag is updated automatically by the CI pipeline after a successful build.

1. Create the Service manifest:

```

cat > environments/dev/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  selector:
    app: etx-app-base-app
  ports:
    - port: 8080
      targetPort: 8080
EOF

```

2. Create the Route manifest:

```

cat > environments/dev/route.yaml <<'EOF'
apiVersion: route.openshift.io/v1
kind: Route
metadata:

```

```

name: etx-app-base-app
namespace: etx-app-dev
spec:
  to:
    kind: Service
    name: etx-app-base-app
  port:
    targetPort: 8080
  tls:
    termination: edge
EOF

```

Commit the Initial Manifests

Commit the development environment manifests and push them to the GitOps repository.

1. Configure your Git identity:

```

git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"

```

2. Retrieve the GitLab administrator credentials:

```

GITLAB_ROOT_PASSWORD=$(oc get secret \
  -n gitlab \
  gitlab-gitlab-initial-root-password \
  -o jsonpath='{.data.password}' | base64 -d)

echo "Root Username: root"
echo "Root Password: ${GITLAB_ROOT_PASSWORD}"

```

3. Commit and push the manifests:

```

git add environments/dev/

git commit -m "feat: add dev environment manifests"

git push origin main

```

Once the changes are pushed, the GitOps repository contains the Kubernetes manifests that Argo CD will use as the desired state for the development environment in the following labs.

Lab 9: Deploy with Argo CD

In this lab, you'll create an Argo CD Application that continuously synchronizes the GitOps repository with your OpenShift cluster. Once configured, Argo CD automatically deploys the

application and keeps the cluster aligned with the desired state stored in Git.

By the end of this lab, you will have:

- Granted Argo CD permission to manage resources in the development project
- Created an Argo CD Application for the development environment
- Verified that the application was synchronized and deployed successfully

Grant Argo CD Permissions

Before Argo CD can deploy applications, its application controller requires permission to manage resources in the **etx-app-dev** project.

1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Grant the required permissions:

```
oc adm policy add-role-to-user admin \
  system:serviceaccount:etx-app-dev:argocd-application-controller \
  -n etx-app-dev
```

This grants the Argo CD Application Controller administrative access within the **etx-app-dev** project so it can create, update, and remove application resources during synchronization.

Create the Argo CD Application

Next, create an Argo CD Application that monitors the GitOps repository and synchronizes the manifests for the development environment.

1. Retrieve the GitLab URL:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: ${GITLAB_URL}"
```

2. Create the Argo CD Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-dev
  namespace: etx-app-dev
```

```
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: etx-app-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF
```

The application continuously watches the `environments/dev` directory in the GitOps repository and automatically synchronizes any changes to the OpenShift cluster.



`CreateNamespace=false` is specified because the `etx-app-dev` namespace already exists.

Verify the Application Synchronization

Confirm that Argo CD has successfully synchronized the application.

1. Check the synchronization status:

```
oc get application etx-app-dev -n etx-app-dev \
  -o jsonpath='{.status.sync.status}{"\n"}'
```

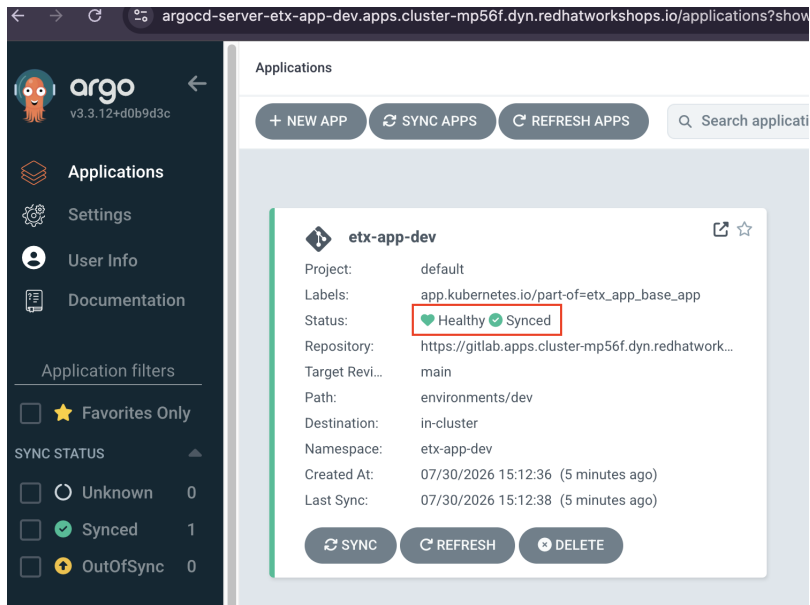
The command should return:

Synced

1. Retrieve the Argo CD dashboard URL:

```
oc get route argocd-server -n etx-app-dev \
  -o jsonpath='{.spec.host}{"\n"}'
```

2. Open the Argo CD dashboard in your browser and verify that the application status is **Synced**.



Verify the Application Deployment

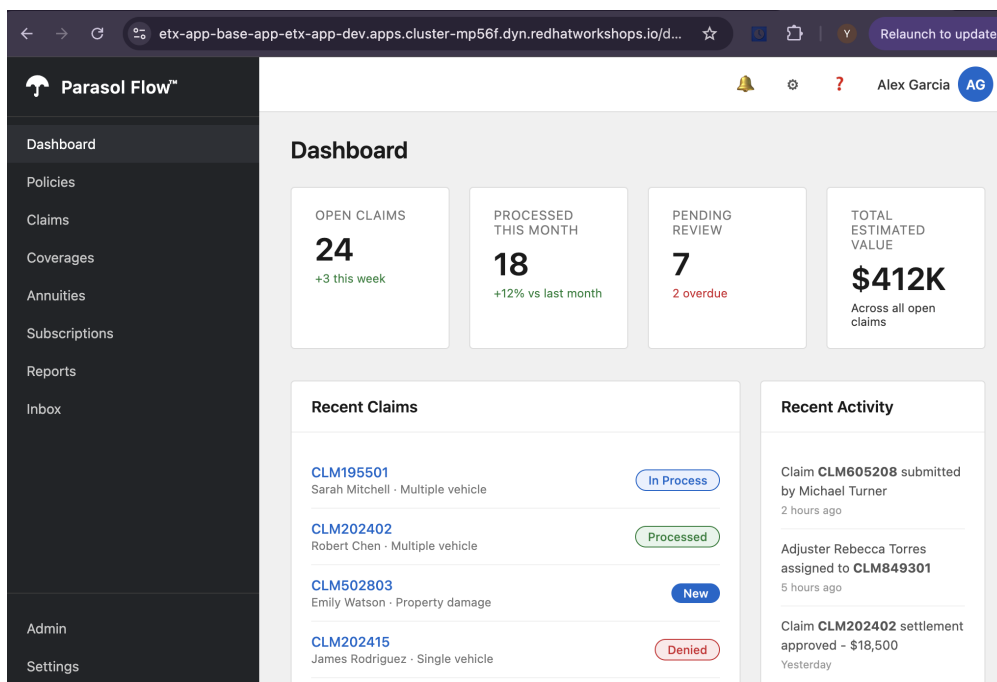
Once synchronization is complete, verify that the application has been deployed successfully.

1. Check that the application pod is running and retrieve the application route:

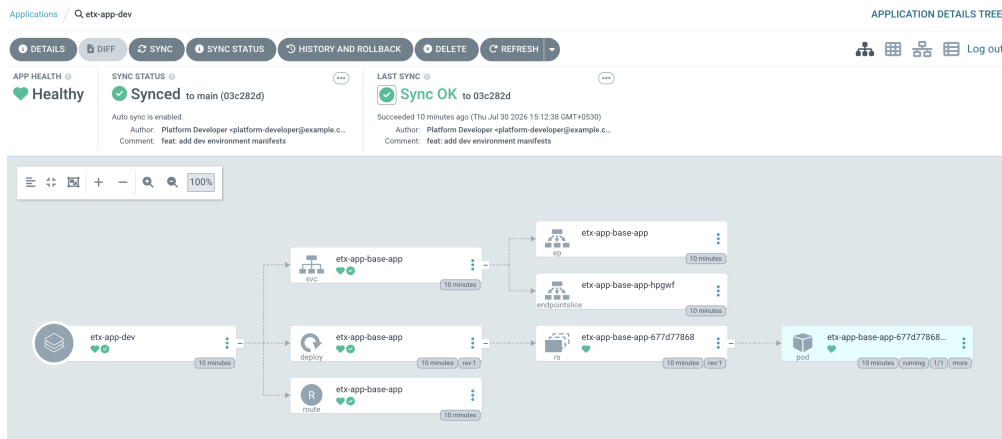
```
oc get pods -n etx-app-dev \
  -l app=etx-app-base-app

oc get route etx-app-base-app -n etx-app-dev \
  -o jsonpath='{.spec.host}{"\n"}'
```

2. Open the application URL in your browser to confirm it is accessible.



3. Return to the Argo CD dashboard and select the application to view its deployed resources and synchronization status.



Lab 10: Verify Observability

In this lab, you'll verify that OpenTelemetry auto-instrumentation is working correctly for the deployed application. You'll confirm that the OpenTelemetry Java agent has been injected into the application pod and that telemetry data is being collected by the OpenTelemetry Collector.

By the end of this lab, you will have:

- Verified that OpenTelemetry auto-instrumentation is enabled for the application
- Confirmed that the OpenTelemetry Java agent was injected into the application pod
- Verified that trace data is being collected by the OpenTelemetry Collector

Verify OpenTelemetry Auto-Instrumentation

In this exercise, you'll confirm that the OpenTelemetry Operator has automatically injected the Java agent into the application deployed through Argo CD.

1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Verify that the application pod contains the OpenTelemetry auto-instrumentation init container:

```
oc get pod -n etx-app-dev \
  -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

The output should include:

```
opentelemetry-auto-instrumentation
```

The presence of this init container confirms that the OpenTelemetry Operator has automatically injected the Java agent into the application pod.

Verify Trace Collection

Next, verify that the application is sending telemetry data to the OpenTelemetry Collector.

1. View the recent logs from the OpenTelemetry Collector:

```
oc logs -n etx-app-dev \
  -l app.kubernetes.io/name=otel-collector-collector \
  --tail=50
```

Review the output and confirm that trace data is being received and processed by the collector.

Once traces appear in the collector logs, you have verified that OpenTelemetry auto-instrumentation is successfully collecting telemetry from the deployed application.

Lab 11: CI to GitOps

In this lab, you'll connect your Tekton CI pipeline to your GitOps repository so that every successful build automatically updates the application image referenced by Argo CD.

By the end of this lab, you will have:

- Created a Tekton Task that updates the GitOps repository with a new image tag
- Extended the CI pipeline to invoke the GitOps update task after a successful image build
- Automated the flow from code commit to GitOps repository update

Create the GitOps Update Task

In this exercise, you'll create a Tekton Task that clones the GitOps repository, updates the application image tag, commits the change, and pushes it back to GitLab.

1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Create the Task:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-repo
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-tag
      type: string
```

```

- name: environment
  type: string
  default: dev
workspaces:
- name: source
steps:
- name: update-image
  image: registry.access.redhat.com/ubi9/ubi:latest
  script: |
    #!/bin/bash
    set -e

    dnf install -y git && dnf clean all

    REPO_URL="$(params.gitops-repo-url)"
    GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^\/]+)/.*|\1|')

    git config --global url."https://etx-platform-
bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"

    git clone "$REPO_URL" /tmp/gitops
    cd /tmp/gitops/environments/${params.environment}

    sed -i "s|image:.*etx-app-base-app:.*|image: image-registry.openshift-
image-registry.svc:5000/etx-app-dev/etx-app-base-app:${params.image-tag}|"
deployment.yaml

    git config user.email "pipeline@openshift.local"
    git config user.name "Tekton Pipeline"

    git add .
    git commit -m "Update etx-app-base-app image to ${params.image-tag}"
    git push
EOF

```

The Task updates the deployment manifest in the GitOps repository with the newly built image tag and pushes the change back to GitLab.

Update the CI Pipeline

Next, extend the existing CI pipeline so it automatically executes the GitOps update task after the container image has been built successfully.

1. Retrieve the GitLab URL:

```

GITLAB_URL="https://${oc get route \
-n gitlab \
-l app=webservice \
-o jsonpath='{.items[0].spec.host}')"

```

```
echo "${GITLAB_URL}"
```

2. Edit the existing pipeline:

```
oc edit pipeline etx-app-base-app-ci -n etx-app-dev
```

3. Under the `spec.tasks` section, add the following task definition:

```
- name: update-gitops
  runAfter:
    - build-image
  taskRef:
    kind: Task
    name: update-gitops-repo
  params:
    - name: gitops-repo-url
      value: SUBSTITUTE-GITLAB-URL-HERE/software-factory/etx-app-gitops.git
    - name: image-tag
      value: $(params.git-revision)
    - name: environment
      value: dev
  workspaces:
    - name: source
      workspace: shared-workspace
```



Replace `SUBSTITUTE-GITLAB-URL-HERE` with the GitLab URL retrieved earlier. If you're recreating the pipeline using a shell heredoc, you can use the `${GITLAB_URL}` variable instead.

Verify the Integration

After saving the pipeline, every successful pipeline execution will automatically update the deployment manifest in the GitOps repository.

When the updated manifest is pushed:

- The GitOps repository receives a new commit with the updated image tag.
- Argo CD detects the repository change automatically.
- Argo CD synchronizes the updated manifest to the cluster.
- The application is redeployed using the newly built container image.

Lab Summary

You have completed the GitOps deployment workflow. Your application platform now provides:

- A dedicated Argo CD instance integrated with Keycloak Single Sign-On
- A GitOps repository containing environment-specific Kubernetes manifests
- Automated application deployment and synchronization with self-healing enabled
- OpenTelemetry auto-instrumentation for application observability
- End-to-end CI/CD integration, where successful Tekton builds automatically update the GitOps repository and trigger application deployment through Argo CD

Application Promotion and GitOps Deployment Workflow

```
<div class="module-card">  
<div class="module-header">
```



What's in this section?

```
</div>
```

In this section, you'll learn how modern application platforms promote applications across environments using a **secure, GitOps-based delivery workflow**.

You'll explore how container images move from development to staging without rebuilding, how Git remains the source of truth for deployment configuration, and how tools such as **OpenShift Pipelines, Quay, Vault, and Argo CD** enable automated, secure, and repeatable application delivery.

Throughout this section, you'll build an application promotion workflow that provides:

- Automated image promotion between environments
- Secure registry credential management
- GitOps-based deployment synchronization
- Multi-cluster application delivery
- Reliable rollback capabilities using Git history and validated images



If you're already familiar with CI/CD pipelines, container registries, and GitOps concepts, feel free to skim this section and continue to [Lab 12: Configure Quay](#). The concepts introduced here provide useful context for understanding how enterprise application platforms manage application releases across environments.

```
</div>
```

Lab Pre-requisites

In this lab, you'll prepare the environment required for multi-cluster application promotion. You'll enroll a runtime cluster with Argo CD, provision the required supporting services, and understand how application promotion moves validated container images from the development environment to staging.

By the end of this lab, you will have:

- Reviewed the multi-cluster application promotion workflow

- Verified the prerequisites for multi-cluster application promotion
- Enrolled a runtime cluster with the factory Argo CD instance
- Provisioned the staging environment and supporting platform services
- Verified that the runtime cluster is registered successfully

Review the Promotion Architecture

In the previous exercises, you built a CI pipeline that automatically builds container images and deploys them to the development environment.

In this lab, you'll prepare the infrastructure required to promote validated application images to a staging environment running on a separate OpenShift cluster.

The promotion workflow performs the following actions:

- Copies container images from the OpenShift internal registry to Quay for cross-cluster access
- Updates the GitOps repository with the new staging image tag
- Allows Argo CD to synchronize the updated manifests to the runtime cluster

Understand the Promotion Workflow

Unlike the development environment, where every successful build is deployed automatically, promotion to staging is an intentional action initiated by the development team.

The promotion workflow consists of the following steps:

Table 1. Promotion Workflow

Step	Component	Action
1	Developer	Approves the promotion by manually triggering the promotion pipeline.
2	GitLab	Starts the promotion pipeline.
3	Promotion Pipeline	Pulls the application image from the OpenShift internal registry using the image SHA.
4	Promotion Pipeline	Pushes the image to Quay using the stable tag.
5	Promotion Pipeline	Updates the environments/stage/deployment.yaml manifest in the GitOps repository.
6	Argo CD	Detects the updated GitOps repository.
7	Argo CD	Synchronizes the updated manifests and deploys the application to the staging cluster.

The promotion workflow follows these principles:

- **Development** uses images stored in the OpenShift internal registry for fast local deployments.
- **Staging** uses images stored in Quay so they can be accessed by multiple OpenShift clusters.

- **Promotion is explicit**, allowing developers to decide when a validated application is ready for staging.
- **GitOps ensures deployments remain version-controlled**, with every staging deployment driven by changes committed to the GitOps repository.

Verify the Prerequisites

Before continuing, ensure you have completed the following exercises:

- [Application CI Pipeline](#)
- [Application Deployment](#)

You should also have:

- A working CI pipeline that builds and pushes application images
- Argo CD managing the application deployment
- Access to Quay using Keycloak Single Sign-On ([platform-admin](#) or [platform-developer](#))
- A runtime OpenShift cluster provisioned through RHDP

Enroll the Runtime Cluster

The promotion workflow requires two OpenShift clusters:

- **Factory Cluster** – Hosts the Software Factory, CI pipelines, and Argo CD.
- **Runtime Cluster** – Hosts the staging and production application environments.

In this exercise, you'll register the runtime cluster with the Argo CD instance running on the factory cluster.

1. Download the enrollment script:

```
curl -o enroll-secondary-cluster.sh \
  https://raw.githubusercontent.com/rh-etx-app-
  platform/etx_app_tooling/main/scripts/enroll-secondary-cluster.sh

chmod +x enroll-secondary-cluster.sh
```

2. Update the configuration variables using the runtime and factory cluster information provided in your RHDP lab environment.

```
export RUNTIME_API="https://api.cluster-<RUNTIME-GUID>.dyn.redhatworkshops.io:6443"
export RUNTIME_USER="admin"
export RUNTIME_PASSWORD="<RUNTIME_cluster_admin_password>"

export FACTORY_API="https://api.cluster-<FACTORY-GUID>.dyn.redhatworkshops.io:6443"
export FACTORY_USER="admin"
```

```
export FACTORY_PASSWORD="<FACTORY_cluster_admin_password>"
```

3. Run the enrollment script:

```
./enroll-secondary-cluster.sh
```

The script automatically performs the following tasks:

- Creates the required ServiceAccount and permissions on the runtime cluster.
- Creates the staging and production namespaces.
- Installs the required platform operators.
- Deploys PostgreSQL, Kafka, OpenTelemetry Collector, and Java auto-instrumentation.
- Registers the runtime cluster with the factory Argo CD instance.

When the script completes successfully, you should see an **Enrollment Complete** message.

Verify the Runtime Cluster Registration

Confirm that the runtime cluster has been registered with Argo CD.

1. Run the following command:

```
oc get secret -n etx-app-dev \
  -l argocd.argoproj.io/secret-type=cluster \
  -o json | \
jq -r '.items[] | .metadata.name + " - " + (.data.name | @base64d)'
```

2. Verify that the output includes both clusters:

```
argocd-default-cluster-config - in-cluster
cluster-staging - staging
```

If both entries are displayed, the runtime cluster has been enrolled successfully and is ready to receive application deployments through Argo CD.

In the next lab, you'll build the promotion pipeline that automatically promotes validated application images from the factory cluster to the staging environment.

Lab 12: Configure Quay

In this exercise, you'll prepare Quay for the image promotion workflow by creating the repository used by the promotion pipeline and storing robot account credentials securely in Vault.

By the end of this exercise, you will have:

- Created the `etx_app` organization and `etx-app-base-app` repository in Quay
- Created a robot account for non-interactive image promotion
- Stored the Quay robot credentials in Vault for use by the promotion pipeline

The promotion pipeline pushes images to Quay at `etx_app/etx-app-base-app` and retrieves authentication credentials from Vault at `secret/registry/quay`.



Interactive users authenticate to Quay using Keycloak Single Sign-On, while CI/CD pipelines use robot accounts for automated, non-interactive authentication.

Create the Quay Organization and Repository

Create the Quay organization and repository that will store promoted application images.

1. Open the **Quay** URL from your lab environment or click the **Quay** tab in your Showroom.
2. Sign in using ETX Keycloak Single Sign-On with one of the lab accounts:
 - `platform-admin / openshift`
 - `platform-developer / openshift`
3. If prompted, remove any hyphens (-) from the suggested username and click **[Confirm Username]**.

Confirm Username

The username `platform-developer` was automatically generated to conform to the Docker CLI guidelines for use as a namespace in Red Hat Quay .

Please confirm the selected username or enter a different username below:

Confirm Username
✔ Username valid

Figure 2. Validate Quay Username

4. Click **[Create New Organization]**.

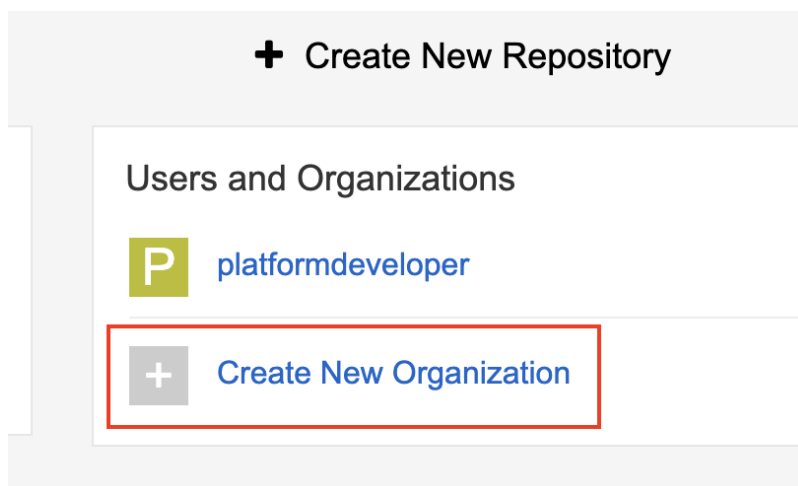


Figure 3. Create Organization

5. Create an organization named `etx_app`.

Figure 4. Create the Organization

- Inside the organization, click **[Create New Repository]**.

Figure 5. New Repository

- Configure the repository:
 - Repository Name:** `etx-app-base-app`
 - Visibility:** Public
- Click **[Create Public Repository]**.

Figure 6. Create Repository



The organization and repository names must match the destination referenced later by the promotion pipeline: `etx_app/etx-app-base-app`.



Although the repository is public, anonymous image pulls are disabled in this Quay deployment. Runtime clusters must authenticate using an image pull secret.

Create a Robot Account

The promotion pipeline requires a robot account to authenticate to Quay without using interactive Keycloak credentials.

- From the `etx_app` organization, click the **Settings** icon.

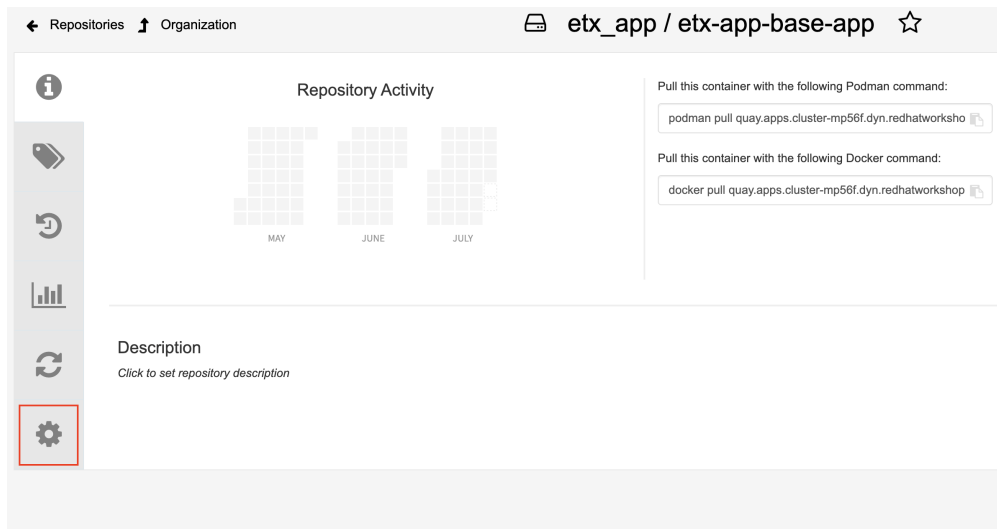


Figure 7. Organization Settings

2. Under **User and Robot Permissions**, open the dropdown and click **[Create Robot Account]**.

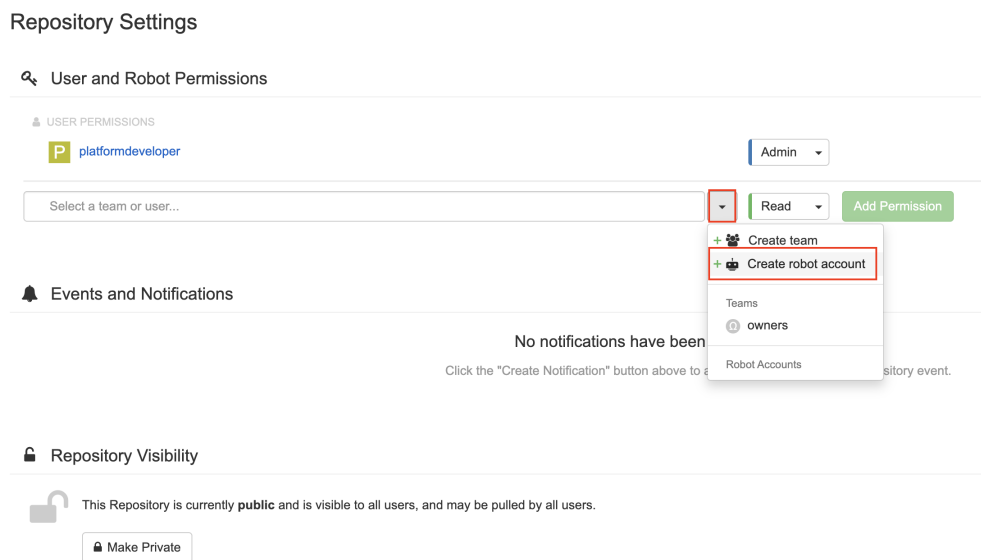


Figure 8. Create Robot Account

3. Create a robot account named **promoter**.

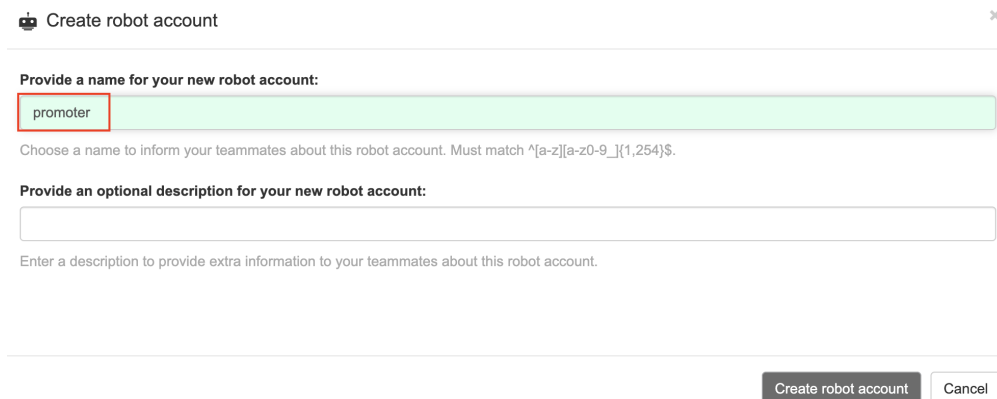


Figure 9. Robot Account Name

4. Grant the robot account **Write** permission to the repository.

Repository Settings

🔍 User and Robot Permissions

USER PERMISSIONS

 platformdeveloper

Admin ▾

 etx_app+promoter

Write ▾

Add Permission

Figure 10. Assign Write Permission

5. Open the newly created robot account (**etx-app+promoter**) and copy the generated credentials:

- **Username:** **etx-app+promoter**
- **Token:** Generated robot token

 Robot Account

 Kubernetes Secret

 rkt Configuration

 Podman Login

 Docker Login

 Docker Configuration

 Mesos Credentials

Credentials for etx_app+promoter

Username & Robot Account:

etx_app+promoter	
4OQXPH4WI0CZOQG2VQMENUG064MOKGHX3YQPX24ZL5PBQEK6H0VK2Y5XCTSRTZS	

Regenerate Token:

Click the link below to regenerate the token for this robot. Note that **all existing logins** of this robot account will become invalid.

[Regenerate Token](#) >

Figure 11. Robot Account Credentials



Save the robot token before leaving the page. It will be required when creating the Vault secret.

Store Quay Credentials in Vault

Store the Quay robot account credentials in Vault so the promotion pipeline can retrieve them securely at runtime.

1. Retrieve the Quay hostname:

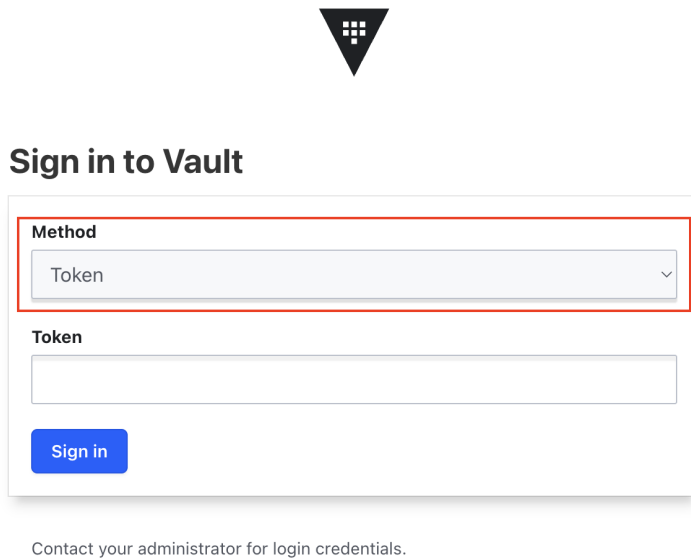
```
oc get route -n quay \
  -l quay-component=quay-app-route \
  -o jsonpath='{.items[0].spec.host}' && echo
```

2. Retrieve the Vault root token:

```
oc get secret etx-vault-bootstrap \
  -n vault \
  -o jsonpath='{.data.root-token}' | base64 -d && echo
```

3. Open the **Vault** URL from your lab environment.

4. Sign in using the **Token** authentication method and paste the token retrieved above.



The image shows the 'Sign in to Vault' form. At the top is the Vault logo. Below it, the title 'Sign in to Vault' is displayed. The form contains a 'Method' dropdown menu with 'Token' selected, a 'Token' text input field, and a blue 'Sign in' button. A red box highlights the 'Method' dropdown. Below the form, a message reads: 'Contact your administrator for login credentials.'

Figure 12. Sign in to Vault

5. From the Vault dashboard, open the **secret/** secrets engine.

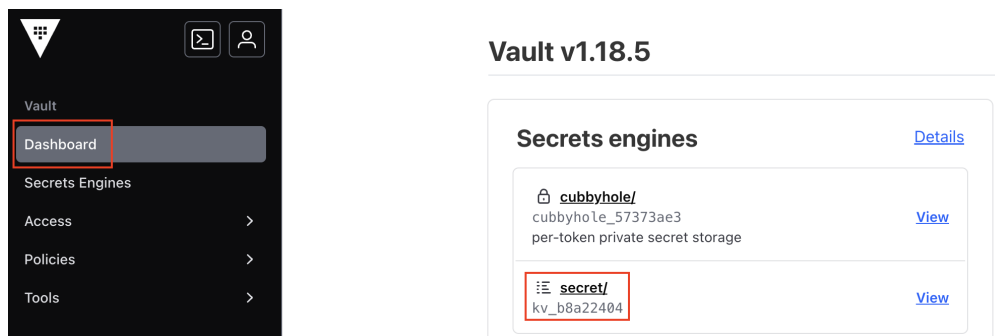


Figure 13. Secret Engine

6. Click [**Create secret**].

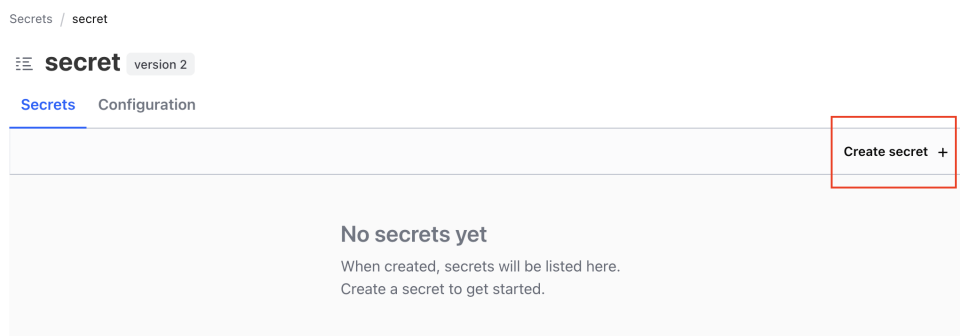


Figure 14. Create Secret

7. Create a secret with the path:

registry/quay

8. Add the following key-value pairs:

- **server:** Quay hostname returned by the earlier command (without `https://`)
- **username:** `etx-app+promoter`
- **password:** Robot token copied from Quay

[illegible]

Figure 15. Create Vault Secret

9. Click **[Save]**.

Secrets / secret / registry / quay

registry/quay



Overview

Secret

Metadata

Paths

Version History

Current version

Create new +

The current version of this secret.

1

Secret age

View metadata →

Current secret version age. Last updated on Jul 31 2026, 3:43:34 PM.

3 minutes

Paths

The paths to use when referring to this secret in API or CLI.

API path

 /v1/secret/data/registry/quay

CLI path

 --mount="secret" "registry/quay"

Figure 16. Completed Secret



The promotion pipeline authenticates to Vault using Kubernetes authentication and retrieves these credentials automatically during execution. No Quay credentials are stored directly in the pipeline definition.

Lab 13: Stage Deployment

In this lab, you'll prepare the staging environment for multi-cluster deployments by creating stage-specific GitOps manifests and configuring the runtime cluster to securely pull application images from Quay.

By the end of this lab, you will have:

- Created GitOps manifests for the staging environment
- Configured the runtime cluster to authenticate with Quay using an image pull secret
- Updated the staging deployment to use Quay-hosted images
- Committed and pushed the staging manifests to the GitOps repository

Prepare the Stage Environment

In this exercise, you'll create a staging environment in the GitOps repository based on the existing development manifests.

1. Clone the GitOps repository (replace **XXXXX** with your lab GUID):

```
git clone https://gitlab.apps.cluster-XXXXX.dyn.redhatworkshops.io/software-
factory/etx-app-gitops.git
cd etx-app-gitops
mkdir -p environments/stage
```

2. Copy the development manifests as the starting point for the staging environment.

```
cp environments/dev/*.yaml environments/stage/
```

3. Retrieve the Quay registry URL that will host promoted application images.

```
QUAY_URL=$(oc get route -n quay \
  -l quay-component=quay-app-route \
  -o jsonpath='{.items[0].spec.host}')

echo "Using Quay URL: $QUAY_URL"
```

Configure Image Pull Access

The runtime cluster must authenticate with Quay before it can pull promoted container images.

1. Log in to the **Service Enablement: App Platform Runtime** cluster using the credentials provided by RHDP.

```
oc login \
```

```
-u <RUNTIME_USERNAME> \  
-p <RUNTIME_PASSWORD> \  
<RUNTIME_API_URL> \  
--insecure-skip-tls-verify
```

2. Create a Quay image pull secret in the **etx-app-staging** namespace using the robot account credentials created in the previous lab.

```
oc create secret docker-registry quay-pull-secret \  
-n etx-app-staging \  
--docker-server=$QUAY_URL \  
--docker-username='etx-app+promoter' \  
--docker-password=$ROBOT_TOKEN
```

3. Switch back to the factory cluster before continuing.

```
oc login $FACTORY_API \  
-u $FACTORY_USER \  
-p $FACTORY_PASSWORD \  
--insecure-skip-tls-verify
```



The runtime cluster uses this image pull secret whenever it pulls application images from Quay.

Create the Stage Deployment Manifest

Create a deployment manifest that references the staging namespace and Quay-hosted application image.

1. Replace the existing deployment manifest with the following configuration.

```
cat > environments/stage/deployment.yaml <<EOF  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: etx-app-base-app  
  namespace: etx-app-staging  
  labels:  
    app: etx-app-base-app  
  annotations:  
    instrumentation.opentelemetry.io/inject-java: "true"  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: etx-app-base-app  
  template:
```

```

metadata:
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  imagePullSecrets:
    - name: quay-pull-secret
  containers:
    - name: etx-app-base-app
      image: ${QUAY_URL}/etx-app/etx-app-base-app:stable
      ports:
        - containerPort: 8080
      resources:
        requests:
          cpu: 500m
          memory: 1Gi
        limits:
          cpu: "2"
          memory: 2Gi
      env:
        - name: POSTGRESQL_HOST
          value: "parasol-db.etx-app-staging.svc.cluster.local"
        - name: POSTGRESQL_DATABASE
          value: "parasol"
        - name: POSTGRESQL_USER
          value: "parasol"
        - name: POSTGRESQL_PASSWORD
          value: "parasol"
        - name: QUARKUS_HIBERNATE_ORM_DATABASE_GENERATION
          value: update

```

EOF

Compared to the development environment, the staging deployment includes several production-oriented changes:

- Deploys to the **etx-app-staging** namespace
- Runs two application replicas for improved availability
- Pulls images from the Quay registry instead of the OpenShift internal registry
- Uses the **quay-pull-secret** to authenticate with Quay

Update and Commit the Stage Manifests

Update the remaining manifests to reference the staging namespace and commit the changes to the GitOps repository.

1. Update the Service and Route manifests.

```
sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/' \
environments/stage/service.yaml \
environments/stage/route.yaml
```

2. Verify that all manifests reference the staging namespace.

```
grep namespace environments/stage/*.yaml
```

3. Commit and push the staging environment manifests.

```
git add -A
git commit -m "feat: add stage environment manifests"
git push origin main
```

Once the changes are pushed, the GitOps repository contains separate deployment definitions for both the development and staging environments, enabling Argo CD to manage each environment independently.

Lab 14: Promotion Pipeline

In this lab, you'll create the Tekton resources required to promote application images from the development environment to the staging environment. The promotion pipeline securely retrieves credentials from Vault, copies images to Quay, and updates the GitOps repository to trigger a deployment.

By the end of this lab, you will have:

- Created a workspace for the promotion pipeline
- Retrieved Quay credentials securely from Vault
- Created a task to copy images from the OpenShift internal registry to Quay
- Created a task to update the staging GitOps manifests automatically

Create the Promotion Pipeline Workspace

The promotion pipeline requires persistent storage for cloning and updating the GitOps repository.

1. Create a PersistentVolumeClaim for the promotion pipeline workspace.

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: promotion-workspace-pvc
  namespace: etx-app-dev
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF
```

Create the Vault Credentials Task

The promotion pipeline retrieves the Quay robot account credentials you stored in Vault rather than embedding secrets directly in the pipeline.

1. Create the `vault-fetch-quay-creds` task.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-quay-creds
  namespace: etx-app-dev
spec:
  results:
    - name: quay-server
    - name: quay-username
    - name: quay-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        microdnf install -y jq && microdnf clean all

        VAULT_ADDR="{vault_url}"
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data "{\"role\":\"pipeline\", \"jwt\":\"${SA_TOKEN}\"}" \
          | jq -r '.auth.client_token')

        CREDS=$(curl -s --header "X-Vault-Token: ${VAULT_TOKEN}" \
          "${VAULT_ADDR}/v1/secret/data/registry/quay")

        echo "${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > $(results.quay-server.path)
        echo "${CREDS}" | jq -r '.data.data.username' \
          | tr -d '\n' > $(results.quay-username.path)
```

```
echo "${CREDS}" | jq -r '.data.data.password' \
  | tr -d '\n' > $(results.quay-password.path)
EOF
```



The task authenticates to Vault using the pipeline ServiceAccount and Kubernetes authentication. Quay credentials are never stored in the pipeline definition.

Create the Image Promotion Task

After retrieving the credentials, the promotion pipeline copies the application image from the OpenShift internal registry to Quay using Skopeo.

1. Create the `copy-image-to-quay` task.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: copy-image-to-quay
  namespace: etx-app-dev
spec:
  params:
    - name: source-image
      type: string
    - name: dest-image
      type: string
    - name: dest-username
      type: string
    - name: dest-password
      type: string
  steps:
    - name: copy
      image: quay.io/skopeo/stable:latest
      script: |
        #!/bin/bash
        set -e

        echo "Copying image from internal registry to Quay..."

        SRC_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        skopeo copy \
          --src-tls-verify=false \
          --dest-tls-verify=false \
          --src-creds="pipeline:${SRC_TOKEN}" \
          --dest-creds="$(params.dest-username):$(params.dest-password)" \
          "docker://$(params.source-image)" \
          "docker://$(params.dest-image)"
EOF
```

EOF

```
echo "Image copied successfully!"
```



Skopeo performs a registry-to-registry image copy without pulling the container image locally, making promotions faster and more efficient.

Create the GitOps Update Task

The final task updates the staging deployment manifest in the GitOps repository with the newly promoted image. Once the change is committed, Argo CD automatically synchronizes the staging environment.

1. Create the `update-gitops-staging` task.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-staging
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-url
      type: string
    - name: git-username
      type: string
      default: "Tekton Pipeline"
    - name: git-email
      type: string
      default: "pipeline@openshift.local"
  workspaces:
    - name: source
  steps:
    - name: update
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        dnf install -y git && dnf clean all

        REPO_URL="$(params.gitops-repo-url)"
        GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^\/]+)/.*|\1|')

        git config --global url."https://etx-platform-
        bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"
```

```

git clone "$REPO_URL" /workspace/gitops
cd /workspace/gitops

sed -i "s|image:.*etx-app-base-app:.*|image: $(params.image-url)|" \
    environments/stage/deployment.yaml

git config user.email "$(params.git-email)"
git config user.name "$(params.git-username)"

git add environments/stage/deployment.yaml

if git diff --cached --quiet; then
    echo "No changes to commit."
else
    git commit -m "Promote to staging: $(params.image-url)"
    git push origin main
    echo "GitOps repository updated successfully!"
fi
EOF

```

Once these tasks are created, you have all the building blocks required for the promotion pipeline. In the next lab, you'll assemble these tasks into a complete Tekton pipeline that automates image promotion from development to the staging environment.

Lab 15: Promotion Pipeline

In this lab, you'll assemble the promotion tasks into a complete Tekton pipeline and verify that it is ready to promote application images from the development environment to the staging environment.

By the end of this lab, you will have:

- Created a Tekton promotion pipeline
- Connected the pipeline tasks into a complete promotion workflow
- Configured the pipeline to copy images to Quay and update the GitOps repository
- Verified the pipeline configuration before running promotions

Create the Promotion Pipeline

In this exercise, you'll combine the tasks created in the previous lab into a single Tekton pipeline.

The promotion pipeline performs the following actions:

- Retrieves Quay credentials securely from Vault
- Copies the selected image from the OpenShift internal registry to Quay
- Updates the staging deployment manifest in the GitOps repository
- Triggers Argo CD to deploy the promoted image to the staging cluster

1. Create the promotion pipeline.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-promotion
  namespace: etx-app-dev
spec:
  params:
    - name: source-image-tag
      type: string
      description: Image tag to promote (e.g., commit SHA)
    - name: dest-image-tag
      type: string
      default: stable
      description: Tag to use in Quay (e.g., stable, v1.0.0)
    - name: gitops-repo-url
      type: string
      default: https://gitlab.apps.cluster-
<GUID>.dyn.redhatworkshops.io/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-quay-creds
      taskRef:
        name: vault-fetch-quay-creds

    - name: copy-to-quay
      runAfter: [fetch-quay-creds]
      taskRef:
        name: copy-image-to-quay
      params:
        - name: source-image
          value: image-registry.openshift-image-registry.svc:5000/etx-app-
dev/etx-app-base-app:${params.source-image-tag}
        - name: dest-image
          value: ${tasks.fetch-quay-creds.results.quay-server}/etx-app/etx-app-
base-app:${params.dest-image-tag}
        - name: dest-username
          value: ${tasks.fetch-quay-creds.results.quay-username}
        - name: dest-password
          value: ${tasks.fetch-quay-creds.results.quay-password}

    - name: update-gitops
      runAfter: [copy-to-quay]
      taskRef:
        name: update-gitops-staging
      params:
        - name: gitops-repo-url
```

```

        value: $(params.gitops-repo-url)
      - name: image-url
        value: $(tasks.fetch-quay-creds.results.quay-server)/etx-app/etx-app-
base-app:$(params.dest-image-tag)
      workspaces:
      - name: source
        workspace: shared-workspace
EOF

```



The pipeline promotes an existing application image. It does not rebuild the application. Instead, it copies a previously tested image from the internal registry to Quay and updates the GitOps repository to trigger deployment.

Verify the Pipeline Configuration

Before running the promotion pipeline, verify that the GitOps repository URL is correct.

1. Retrieve the GitLab URL dynamically.

```

GITLAB_URL="https://$(oc get route -n gitlab \
-l app=webservice \
-o jsonpath='{.items[0].spec.host}')"

echo "GitLab URL: $GITLAB_URL"

```

Use this URL when supplying the `gitops-repo-url` parameter if it differs from the default value configured in the pipeline.

Once the pipeline has been created successfully, you're ready to execute a promotion and verify that the application is deployed automatically to the staging cluster.

Lab 16: Test Promotion

In this lab, you'll verify the complete promotion workflow by manually triggering the promotion pipeline and confirming that it copies a container image to Quay and updates the GitOps repository for the staging environment.

By the end of this lab, you will have:

- Selected a container image built by the CI pipeline for promotion
- Triggered the Tekton promotion pipeline
- Verified that the image was copied from the internal registry to Quay
- Confirmed that the GitOps repository was updated with the promoted image

Retrieve an Image Tag

The promotion pipeline requires the tag of an image previously built by the CI pipeline.

1. List the available image tags:

```
oc get imagestream etx-app-base-app -n etx-app-dev -o json | \
jq -r '.status.tags[].tag' | head -5
```

2. Note one of the commit SHA tags from the output. You will use it when creating the PipelineRun.

Refresh the Vault Kubernetes Authentication Token

Before running the promotion pipeline, refresh the Vault Kubernetes authentication configuration.

1. Run the following command:

```
oc exec -n vault etx-vault-0 -- sh -c \
"VAULT_ADDR=\"https://$(oc get route -n vault etx-vault -o
jsonpath='{.spec.host}')\" \
VAULT_TOKEN=\"$(oc get secret -n vault etx-vault-bootstrap -o
jsonpath='{.data.root-token}' | base64 -d)\" \
vault write auth/kubernetes/config \
token_reviewer_jwt=\"$(oc create token etx-vault -n vault --duration=24h)\" \
kubernetes_host='https://kubernetes.default.svc' \
kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt"
```



This refreshes the Kubernetes Token Reviewer JWT used by Vault. In a production environment, this configuration is typically automated and does not require manual intervention.

Trigger the Promotion Pipeline

Create a PipelineRun to promote the selected image to the staging environment.

1. Create the PipelineRun, replacing `<COMMIT_SHA>` with one of the image tags from the previous step:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
```

```

params:
  - name: source-image-tag
    value: <COMMIT_SHA>
  - name: dest-image-tag
    value: stable
  - name: gitops-repo-url
    value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
workspaces:
  - name: shared-workspace
    persistentVolumeClaim:
      claimName: promotion-workspace-pvc
taskRunTemplate:
  serviceAccountName: pipeline
EOF

```

Verify the Promotion

Confirm that the promotion pipeline completed successfully.

1. Watch the PipelineRun status:

```
oc get pipelinerun -n etx-app-dev -w
```

Press **Ctrl** + **C** once the PipelineRun reaches the **Succeeded** state.

The promotion pipeline performs the following actions:

- Retrieves the Quay robot account credentials from Vault
- Copies the selected image from the OpenShift internal registry to Quay
- Updates the staging deployment manifest in the GitOps repository with the promoted image

At this point, the GitOps repository references the promoted image, allowing Argo CD to detect the change and deploy the updated application to the staging environment.

Lab 17: Stage Deployment

In this lab, you'll deploy the staging application by creating an Argo CD Application that targets the secondary cluster. Argo CD will continuously synchronize the staging environment with the manifests stored in the GitOps repository.

By the end of this lab, you will have:

- Created an Argo CD Application for the staging environment
- Configured Argo CD to deploy to the secondary cluster
- Verified that the staging application is synchronized successfully
- Confirmed that the staging deployment is visible in the Argo CD dashboard

Create the Staging Application

In this exercise, you'll create an Argo CD Application that deploys the staging manifests from the GitOps repository to the runtime cluster.

1. Retrieve the GitLab repository URL:

```
GITLAB_URL="https://$(oc get route -n gitlab \
-l app=web-service \
-o jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: ${GITLAB_URL}"
```

2. Create the Argo CD Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-stage
  namespace: etx-app-dev
  labels:
    app.kubernetes.io/part-of: etx_app_base_app
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/stage
  destination:
    name: staging
    namespace: etx-app-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
```



The **staging** destination refers to the secondary cluster that was enrolled with Argo CD in the earlier lab.

Verify the Staging Deployment

Confirm that Argo CD has synchronized the staging application successfully.

1. Check the synchronization status:

```
oc get application etx-app-stage -n etx-app-dev \
-o jsonpath='{.status.sync.status}{"\n"}'
```

The output should be:

Synced

2. Open the Argo CD dashboard.
3. Verify that the following applications are listed:
 - **etx-app-dev** – Application deployed to the primary (development) cluster
 - **etx-app-stage** – Application deployed to the secondary (staging) cluster
4. Select the **etx-app-stage** application to review its synchronization status, managed resources, and deployment health.



Because the staging cluster is registered with Argo CD, you can monitor and manage the remote deployment directly from the Argo CD dashboard without logging in to the secondary cluster.

Lab 18: Verify Promotion

In this lab, you'll validate the complete GitOps promotion workflow by verifying that the application is healthy in the development environment and that the promotion pipeline successfully synchronized the staging environment through Argo CD.

By the end of this lab, you will have:

- Reviewed the end-to-end GitOps promotion workflow
- Verified that the development application is healthy
- Confirmed that the GitOps repository references the promoted image
- Verified that Argo CD synchronized the staging application successfully

Understand the Promotion Workflow

In this exercise, you'll review how a container image moves from the development environment to the staging environment using Tekton, Quay, Git, and Argo CD.

The promotion workflow consists of the following steps:

1. The CI pipeline builds the application and pushes a container image to the OpenShift internal registry using the commit SHA as the image tag.
2. The promotion pipeline copies the validated image to the Quay registry and assigns it the **stable** tag.
3. The promotion pipeline updates the **environments/stage/deployment.yaml** manifest in the GitOps

repository with the promoted image.

4. Argo CD detects the Git commit and automatically synchronizes the updated manifests to the staging cluster.

This GitOps workflow provides several advantages:

- **Version control** – All deployment changes are stored and tracked in Git.
- **Auditability** – Every deployment is associated with a Git commit, providing a complete deployment history.
- **Rollback capability** – Reverting a Git commit restores a previous application version.
- **Multi-cluster consistency** – The same Git repository can be used to deploy applications across multiple clusters.

Verify the Development Deployment

Confirm that the development environment is healthy and that the application is running correctly.

1. Verify the deployment status:

```
# Check Argo CD application status
oc get application etx-app-dev -n etx-app-dev \
  -o jsonpath='{.status.sync.status}: {.status.health.status}{"\n"}'

# Check application pods
oc get pods -n etx-app-dev -l app=etx-app-base-app

# Verify the application health endpoint
curl https://$(oc get route etx-app-base-app -n etx-app-dev \
  -o jsonpath='{.spec.host}')/q/health

# Verify OpenTelemetry auto-instrumentation
oc get pod -n etx-app-dev -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

You should observe:

- Argo CD reports **Synched: Healthy**
- Application pods are in the **Running** state
- The health endpoint returns a successful response
- The **opentelemetry-auto-instrumentation** init container is present



Common Issues

- **ImagePullBackOff** — Verify that the CI pipeline successfully built and pushed the application image.
- **CrashLoopBackOff** — Review the application logs and confirm that required

services such as PostgreSQL and Kafka are available.

- **Argo CD Out of Sync** — Synchronize the application manually from the Argo CD dashboard or verify the GitOps repository changes.
- **Health endpoint unavailable** — Ensure that all supporting services are running and the application has started successfully.

Verify the Staging Promotion

Confirm that the promotion pipeline updated the GitOps repository and that Argo CD synchronized the staging environment.

1. Verify that the staging deployment manifest references the promoted image:

```
cd ~/workshop/etx-app-gitops

git pull origin main

grep image: environments/stage/deployment.yaml
```

The deployment manifest should reference the Quay image using the **stable** tag.

2. Verify the staging application status:

```
oc get application etx-app-stage -n etx-app-dev \
-o jsonpath='{.status.sync.status}{"\n"}'
```

The output should be:

```
Synced
```

3. Open the Argo CD dashboard.
4. Select the **etx-app-stage** application and verify that:
 - The application status is **Synced** and **Healthy**
 - The deployment references the promoted image stored in Quay

At this point, you have validated the complete GitOps promotion workflow. A successful promotion updates the GitOps repository, and Argo CD automatically deploys the new application version to the staging cluster.

Lab 19: Promotion Automation

In this lab, you'll explore different approaches for controlling application promotion from development to staging, ranging from manual execution to approval-based and Git-driven workflows.

By the end of this lab, you will have:

- Understood different promotion strategies for application releases
- Added an optional manual approval step to a Tekton pipeline
- Learned how GitLab merge requests can be used to control promotions

Understand Promotion Strategies

The promotion pipeline you created in the previous labs can be triggered in several ways depending on your team's release process.

Option 1: Manual Pipeline Execution

The current implementation requires a user to manually start the promotion pipeline after verifying that the application is ready for staging.

This approach is suitable for:

- Small development teams
- Infrequent application releases
- Release processes requiring manual verification

Option 2: Add a Manual Approval Step

Instead of manually starting the pipeline, you can allow the pipeline to start automatically and pause until someone explicitly approves the promotion.

1. Create a Tekton task that waits for manual approval:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: manual-approval
  namespace: etx-app-dev
spec:
  description: |
    Pauses pipeline execution until manual approval.
    Requires operator intervention to continue.
  steps:
    - name: wait-for-approval
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        echo "" Pipeline paused - waiting for manual approval"
        echo "Run: oc patch taskrun <taskrun-name> -p
'{"spec\":{\"status\":\"Succeeded\"}}'"
        sleep infinity
```

The task pauses the pipeline until an operator manually approves the corresponding TaskRun.

2. Update the promotion pipeline by adding this task as the first task in the pipeline before the image promotion steps.

This allows the pipeline to begin automatically while still requiring explicit approval before promoting an image to the staging environment.

Option 3: Trigger Promotion from GitLab

Another common approach is to promote applications through a Git-based approval workflow.

In this model:

1. Create a dedicated promotion branch in the application repository.
2. Open a Merge Request for the promotion.
3. Request review and approval from the appropriate team members.
4. Configure a GitLab webhook to trigger the promotion pipeline after the Merge Request is merged.

This approach provides several benefits:

- Promotion decisions are reviewed before deployment.
- Git maintains a complete audit history of approvals.
- Teams collaborate using a familiar pull request workflow.
- Promotion becomes part of the standard GitOps development process.

Lab 20: Rollback

In this lab, you'll learn two common approaches for rolling back an application deployment in a GitOps environment when a release needs to be reverted.

By the end of this lab, you will have:

- Rolled back a deployment by reverting a Git commit
- Promoted a previously built application image back to staging
- Understood how GitOps enables safe and repeatable rollbacks

Roll Back Using Git

The recommended rollback approach is to restore a previous version of the deployment manifests in the GitOps repository.

1. Navigate to your local GitOps repository:

```
cd ~/workshop/etx-app-gitops
```

2. Review the commit history for the staging deployment manifest:

```
git log environments/stage/deployment.yaml
```

3. Revert the desired commit and push the change:

```
git revert <commit-hash>  
git push origin main
```

Once the commit is pushed, Argo CD automatically detects the change and synchronizes the staging environment to the previous application version.

Roll Back by Re-Promoting a Previous Image

If you know the image tag of a previously validated application version, you can rerun the promotion pipeline using that image.

1. Create a new PipelineRun using the previous image tag:

```
oc create -f - <<EOF  
apiVersion: tekton.dev/v1  
kind: PipelineRun  
metadata:  
  generateName: etx-app-promotion-  
  namespace: etx-app-dev  
spec:  
  pipelineRef:  
    name: etx-app-promotion  
  params:  
    - name: source-image-tag  
      value: <PREVIOUS_SHA>  
    - name: dest-image-tag  
      value: stable  
  workspaces:  
    - name: shared-workspace  
      persistentVolumeClaim:  
        claimName: promotion-workspace-pvc  
  taskRunTemplate:  
    serviceAccountName: pipeline  
EOF
```

The promotion pipeline copies the selected image to Quay, updates the GitOps repository, and Argo CD automatically deploys the previous application version to the staging environment.



In a GitOps workflow, Git remains the source of truth for deployments. Whether you revert a Git commit or re-promote an earlier application image, Argo CD continuously reconciles the cluster to match the desired state stored in the GitOps repository.

Lab Summary

□ Lab Summary

You have completed the Promotion Pipeline exercise and established a secure, GitOps-based application promotion workflow.

You now have the following capabilities:

- **Automated Image Promotion** – Application images can be promoted from the internal registry to Quay without rebuilding.
- **GitOps-Based Deployments** – Staging deployments are managed through Git repositories and synchronized by Argo CD.
- **Secure Credential Management** – Registry credentials are securely retrieved and managed using Vault.
- **Multi-Cluster Deployment Support** – Application versions can be promoted across different OpenShift environments.
- **Rollback Capability** – Previous application versions can be restored using Git history or previously validated images.

Key Takeaways

Promotion Does Not Mean Re-Build

The promotion process moves the same validated container image between environments.

- The same container image identified by its SHA is promoted from development to staging.
- No application rebuild occurs between environments.
- Configuration changes are applied separately from the application artifact.
- This ensures that the application you test is exactly the application you deploy.

GitOps Ensures Consistency

Git remains the source of truth for application deployments.

- All deployment state is stored and versioned in Git.
- Changes are fully auditable and reversible.
- Argo CD continuously synchronizes the cluster state with the desired state defined in Git.
- Drift detection ensures that environments remain consistent.

Multi-Cluster Architecture

The promotion workflow supports application delivery across multiple OpenShift environments.

- Development uses the internal registry for fast local image builds and testing.
- Staging uses Quay for a trusted, externally accessible image registry.
- Argo CD provides centralized deployment management across clusters.